West University of Timişoara

Faculty of Informatics

Department of Computer Science

Six Sorts, Seven Scales

An Empirical Study of Classical Sorting Algorithms Across Sizes and Input Shapes

Maria Maiorescu

maria.maiorescu06@e-uvt.ro

April 20, 2026

Abstract

Sorting is one of the first problems every computer-science student learns, and also one of the hardest to analyse well in practice. Textbook complexity bounds tell us a lot, but they hide constants, cache effects, and the shape of real input data. This paper takes a practical look at what actually happens when we run these algorithms.

I implemented six classical sorting algorithms in C — Bubble Sort, Insertion Sort, Selection Sort, Merge Sort, Quick Sort, and Radix Sort — and measured their CPU time on arrays ranging from 10 to one million integers. Each array was generated in several shapes: fully random, fully reversed, and partially sorted at five different levels. For small arrays every timing is the average of one million runs; for larger ones the number of repetitions was reduced until each measurement remained trustworthy.

The numbers confirm most of the theory, but they also show effects that pure complexity analysis misses: insertion sort becomes surprisingly competitive once the input is almost in order, the two quadratic algorithms lose any chance of usefulness past ten thousand elements, quick sort with median-of-three can still behave badly on certain structured inputs, and radix sort ends up the fastest method at every large size tested.

Contents

1 Introduction **3**
1.1 Motivation and Context . . . 3
1.2 Existing Solutions and Their Limitations . . . 3
1.3 Outline of This Work . . . 3

2 A Formal View of Sorting and the Six Chosen Algorithms **4**
2.1 The Sorting Problem . . . 4
2.2 Algorithmic Complexity . . . 4

3 Turning the Algorithms into a C Benchmark **4**
3.1 Program Architecture . . . 4
3.2 Data Structures . . . 5
3.3 Data Generators . . . 5
3.4 Timing Methodology . . . 5
3.5 Key Implementation Details . . . 6
3.6 User Manual: Building and Running the Program . . . 7

4 Experimental Study **8**
4.1 Experimental Setup . . . 8
4.2 Results: Small Arrays ($n \leq 100$) . . . 8
4.3 Results: Medium Arrays ($n = 1\,000$) . . . 9
4.4 Results: Large Arrays ($n = 10\,000$ to $n = 1\,000\,000$) . . . 9
4.5 Analysis and Discussion . . . 10

5 Related Work **11**

6 Conclusions and Future Work **11**
6.1 Summary . . . 11
6.2 Problems Encountered . . . 11
6.3 Future Work . . . 12

1 Introduction

1.1 Motivation and Context

Sorting is everywhere — inside database query plans, compilers, search engines, graphics pipelines, and almost every program that scans an ordered collection. Because it is so common, any improvement pays off many times. Sorting is also one of the most studied topics in algorithmics, from Knuth's classical treatment [1] to modern work on cache-aware and parallel sorts.

There is, however, a clear gap between theory and practice. Asymptotic complexity describes how the running time grows when the input becomes arbitrarily large, but it says nothing about constants, cache behaviour, or what happens when the data is not random. Real inputs are rarely random: log files arrive almost sorted by time, priority queues can appear reversed, and database columns often contain long runs of repeats. An algorithm that wins on paper may therefore lose on such data, and vice versa. The goal of this paper is to explore that gap in a controlled way.

1.2 Existing Solutions and Their Limitations

The six algorithms picked for the study are the ones taught in a standard Algorithms and Data Structures course, and they cover the three main families of comparison-based sorting plus one non-comparison method:

- **Bubble Sort** and **Selection Sort** are simple $\mathcal{O}(n^2)$ methods. They are easy to explain but rarely the right choice outside of teaching.
- **Insertion Sort** is also quadratic in the worst case, but it drops to $\mathcal{O}(n)$ on input that is already close to sorted. Hybrid libraries like Timsort exploit exactly this property.
- **Merge Sort** guarantees $\mathcal{O}(n \log n)$ time regardless of the input, at the cost of $\mathcal{O}(n)$ extra memory.
- **Quick Sort** is usually the fastest comparison-based sort in practice, but its worst case is still $\mathcal{O}(n^2)$. Good pivot selection is what keeps it out of that corner.
- **Radix Sort** escapes the $\Omega(n \log n)$ lower bound by using the digits of the keys rather than comparisons. It runs in $\mathcal{O}(d \cdot n)$, but works only for keys that can be decomposed into digits.

Most introductory comparisons stop at random data and a single array size. The limitation is obvious: real-world inputs rarely look like that, and the relative ranking of algorithms can change a lot once the input has structure.

1.3 Outline of This Work

Section 2 gives the formal definition and a short summary of each algorithm's complexity. Section 3 describes the C implementation, the data generators, the timing layer, and how to compile and run the program on Linux and Windows. Section 4 presents the measurements and discusses what stands out. Section 5 places the results next to prior studies and similar student projects. Section 6 lists the problems encountered and sketches future work; the bibliography closes the paper.

The benchmarking program — every sorting routine, every data generator, and the whole timing framework — was written by me from scratch in C. The choice of input sizes, the set of structural variants, and the averaging strategy are my own, and so are the analysis and the writing. Whenever an idea comes from a published source, it is cited explicitly. A reader who only wants the high-level story can focus on the abstract, Section 4.5, and the conclusion.

2 A Formal View of Sorting and the Six Chosen Algorithms

2.1 The Sorting Problem

Definition 1 (Sorting Problem)**.** *Let $A = \langle a_1, a_2, \ldots, a_n \rangle$ be a sequence of n elements drawn from a totally ordered set $(S, \leq)$. A* sorting algorithm *produces a permutation $A' = \langle a_{\pi(1)}, a_{\pi(2)}, \ldots, a_{\pi(n)} \rangle$ such that*

$$a_{\pi(1)} \leq a_{\pi(2)} \leq \cdots \leq a_{\pi(n)}.$$

For this study the elements are non-negative integers, so $S = \mathbb{N}$ and $\leq$ is the usual order. The keys and the full records coincide, which keeps the discussion focused on pure sorting performance rather than on record movement costs.

2.2 Algorithmic Complexity

Table 1 collects the best, average, and worst-case time complexities, together with the extra memory each method needs. The numbers are standard results and can be found, with proofs, in Knuth [1] and CLRS [2].

Table 1: Theoretical time and space complexity of the six algorithms studied.

Algorithm	**Best**	**Average**	**Worst**	**Space**
Bubble Sort	$\mathcal{O}(n)$	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	$\mathcal{O}(1)$
Insertion Sort	$\mathcal{O}(n)$	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	$\mathcal{O}(1)$
Selection Sort	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	$\mathcal{O}(1)$
Merge Sort	$\mathcal{O}(n \log n)$	$\mathcal{O}(n \log n)$	$\mathcal{O}(n \log n)$	$\mathcal{O}(n)$
Quick Sort	$\mathcal{O}(n \log n)$	$\mathcal{O}(n \log n)$	$\mathcal{O}(n^2)$	$\mathcal{O}(\log n)$
Radix Sort (LSD)	$\mathcal{O}(dn)$	$\mathcal{O}(dn)$	$\mathcal{O}(dn)$	$\mathcal{O}(n + d)$

For the LSD base-10 radix sort used here, $d = \lfloor \log_{10} \max(A) \rfloor + 1$. Since the generated values are bounded by n, d never grows past 7 in the experiments.

A classical result that frames the whole comparison is the following.

Proposition 1 (Lower bound for comparison-based sorting)**.** *Any comparison-based sorting algorithm needs $\Omega(n \log n)$ comparisons in the worst case [2].*

This bound is what makes Merge Sort and Quick Sort (in the average case) asymptotically optimal among methods that only compare keys. Radix Sort sidesteps the bound because it reads digits directly rather than comparing whole keys, which is why it can be linear in n for fixed-width integers.

3 Turning the Algorithms into a C Benchmark

3.1 Program Architecture

The whole experiment lives in a single C99 source file, `algorithms.c`. I deliberately kept it flat rather than splitting it across headers, because the goal was to have one self-contained program that could be read top to bottom. Internally it is organised in four layers:

1. **Data generation** — functions that allocate an `int` array and fill it according to a requested shape (random, reversed, or partially sorted).
2. **Sorting** — six sorting routines, each working in place on an `int*` buffer.
3. **Timing** — thin wrappers around `clock()` that return elapsed milliseconds.

4. **Benchmarking** — two orchestration functions (`analyzeSortAlg` and `analyzeAlgo`) that loop over repetitions, record the averages, and append them to `results.csv`.

3.2 Data Structures

The only data structure that matters for the comparison is the plain C array of `int`. Each run allocates a fresh array, fills it with the required pattern, sorts it, and frees it, so no state ever leaks between runs. Two sorting routines need auxiliary storage:

- Merge Sort builds two temporary sub-arrays `L` and `R` at every merge step and frees them at the end of the step.
- Radix Sort allocates one output buffer of size n per digit pass, which is reused across the digits of one sort.

3.3 Data Generators

The three input shapes are produced by dedicated functions. The partially-sorted generator starts from the sorted sequence $\langle 0, 1, \ldots, n-1 \rangle$ and performs a controlled number of random swaps:

```
int* generatePartiallySortedIntArray(unsigned long size, int factor) {
    int* array = malloc(sizeof(int) * size);
    for (unsigned long i = 0; i < size; i++) array[i] = (int)i;

    unsigned long swaps = size / factor;  // fewer swaps => more sorted
    for (unsigned long k = 0; k < swaps; k++) {
        unsigned long i = rand() % size;
        unsigned long j = rand() % size;
        int tmp = array[i]; array[i] = array[j]; array[j] = tmp;
    }
    return array;
}
```

Listing 1: Partially-sorted generator (simplified).

Higher `factor` values give fewer swaps and therefore arrays that are closer to sorted. The five values used in the experiments are $10, 30, 50, 70, 90$, corresponding to roughly $10\%, 3.3\%, 2\%, 1.4\%$, and 1.1% of positions disturbed. The reversed generator fills the array with $n-1, n-2, \ldots, 0$, and the random generator uses `rand() % (size + 1)` with a single seed at program start.

3.4 Timing Methodology

Time is measured with `clock()`, which reports CPU time consumed by the process. The elapsed milliseconds are

$$t = \frac{\texttt{clock_end} - \texttt{clock_start}}{\texttt{CLOCKS_PER_SEC}} \times 1000.$$

Wrapping that in two tiny helpers keeps the sorting code clean:

```
clock_t startOperationTimer(void) { return clock(); }

double stopOperationTimerMs(clock_t startTime) {
    clock_t endTime = clock();
    return ((double)(endTime - startTime) * 1000.0) / CLOCKS_PER_SEC;
}
```

Listing 2: Start/stop timing helpers.

Because very small sorts finish well below one millisecond, for $n \in \{10, 50, 100\}$ the measurement is the average of 10^6 independent runs. For $n = 1\,000$ the average is over 100 runs. For $n \geq 10\,000$ a single run is enough to be well above the clock resolution, and for $n = 10\,000$ I still average over 10 runs to reduce noise.

3.5 Key Implementation Details

Two optimisations are worth showing in code, because they directly influence the measured times.

Bubble Sort with early exit. The flag is what gives Bubble Sort its $\mathcal{O}(n)$ best case. Without it, the algorithm cannot notice that the array is already sorted.

```
void bubbleSort(int* array, unsigned long size) {
    for (unsigned long i = 0; i < size - 1; i++) {
        int swapped = 0;
        for (unsigned long j = 0; j < size - 1 - i; j++) {
            if (array[j] > array[j + 1]) {
                int tmp = array[j];
                array[j] = array[j + 1];
                array[j + 1] = tmp;
                swapped = 1;
            }
        }
        if (!swapped) return;    // already sorted, stop early
    }
}
```

Listing 3: Bubble Sort with the `swapped` flag.

Quick Sort with median-of-three pivot. The pivot is picked as the median of the first, middle, and last elements of the current range. This mitigates the classical worst case on already-sorted or fully-reversed data.

```
long partitionQuickSort(int* array, long low, long high) {
    long mid = low + (high - low) / 2;
    // move median of (array[low], array[mid], array[high]) into array[
        high]
    if (array[mid] < array[low])  swapInts(&array[low], &array[mid]);
    if (array[high] < array[low]) swapInts(&array[low], &array[high]);
    if (array[high] < array[mid]) swapInts(&array[mid], &array[high]);
    swapInts(&array[mid], &array[high]);    // pivot now at array[high]
    /* ... standard Lomuto partition around array[high] ... */
}
```

Listing 4: Median-of-three pivot selection (sketch).

Radix digit pass. Radix Sort (LSD, base 10) runs one counting-sort pass per digit, using a 10-slot counting array and one auxiliary buffer of size n. Stability is guaranteed by scanning the input from right to left during the final fill:

```
void countSortByDigit(int* array, unsigned long size, int exp) {
    int* output = malloc(size * sizeof(int));
    int count[10] = {0};
```

```c
    for (unsigned long i = 0; i < size; i++) count[(array[i] / exp) %
        10]++;
    for (int d = 1; d < 10; d++) count[d] += count[d - 1];
    for (long i = (long)size - 1; i >= 0; i--) {
        int d = (array[i] / exp) % 10;
        output[--count[d]] = array[i];
    }
    for (unsigned long i = 0; i < size; i++) array[i] = output[i];
    free(output);
}
```

Listing 5: One digit pass of LSD radix sort.

Radix Sort scope. The radix sort operates in base 10 and only handles non-negative integers. The generators guarantee that constraint, so I did not add sign handling.

3.6 User Manual: Building and Running the Program

The program has no command-line arguments. Its output consists of two things: one line printed for every measurement on standard output, and the same line appended to `results.csv` in the current working directory. The CSV columns are

```
Elements, Runs, Algorithm, Variant, Elapsed time(ms)
```

On Linux (or macOS). Any recent `gcc` or `clang` works. A typical workflow on Debian or Ubuntu is:

```bash
# 1. Install a compiler if needed (Debian/Ubuntu):
sudo apt update
sudo apt install build-essential

# 2. Compile with optimisation enabled:
gcc -O2 -std=c99 -o algorithms algorithms.c -lm

# 3. Run (results.csv will be written in the current folder):
./algorithms

# 4. Inspect the results:
head -n 5 results.csv
wc -l results.csv        # count how many rows were produced
```

On macOS, replace the install step with `xcode-select --install`; the rest of the commands are identical. On Fedora or a RHEL-like distribution, use `sudo dnf install gcc make`.

The `-O2` flag matters: without it, the quadratic algorithms slow down by a factor of two or three. Re-running the $n = 100$ block with `-O0` increased Merge Sort's per-sort time from 0.010 ms to about 0.028 ms while Radix Sort only moved from 0.005 ms to 0.009 ms, which narrows the gap between the two noticeably. To start from a fresh CSV, `rm -f results.csv && ./algorithms` is enough, since the program re-creates the file on start.

On Windows. There are three practical options, from the most Unix-like to the most native.

Option A — Windows Subsystem for Linux. WSL runs a real Linux kernel on Windows, so the commands are exactly the ones from the Linux section. After `wsl --install -d Ubuntu` (once, from an elevated PowerShell), entering `wsl` drops you into an Ubuntu shell where the Linux workflow applies as-is.

Option B — MSYS2 / MinGW-w64 (native Windows binary, GCC toolchain).

```
# In the MSYS2 MinGW64 shell, install a compiler:
pacman -Syu
pacman -S mingw-w64-x86_64-gcc

# Compile and run:
gcc -O2 -std=c99 -o algorithms.exe algorithms.c -lm
./algorithms.exe
```

Option C — Microsoft Visual C++ from a "x64 Native Tools Command Prompt".

```
rem Compile with the MSVC command-line driver:
cl /O2 /Fe:algorithms.exe algorithms.c

rem Run:
algorithms.exe

rem Peek at the CSV (PowerShell style):
powershell Get-Content results.csv -TotalCount 5
```

A few Windows-specific things that are easy to trip over: the shell prompt symbol is `>` rather than `$`; `CLOCKS_PER_SEC` is defined on both platforms but its numerical value differs (1000 on typical Windows runtimes, 1 000 000 on most Linux ones), which is why the code divides by it explicitly instead of hard-coding a constant; and MSVC does not accept the POSIX `-lm` flag because its math functions live in the standard CRT by default.

Whichever route is chosen, the expected first lines of `results.csv` are `Elements, Runs, Algorithm, Variant, Elapsed time(ms)` followed by one row per (algorithm, variant, size) triple.

Adding more sizes. To run sizes beyond the defaults, it is enough to uncomment the corresponding `analyzeAlgo` calls in `main()`. A warning from experience: running $n = 10^6$ on the three quadratic algorithms takes several hours of CPU time, so I recommend first producing the results for the fast algorithms and only then launching the slow ones as a separate batch.

4 Experimental Study

4.1 Experimental Setup

All measurements were taken on a personal laptop running a Unix-like operating system, with the program compiled using `gcc -O2`. While the large-size experiments were running, I avoided launching any CPU-intensive background tasks. The random generator is seeded exactly once, at program start, via `srand(time(NULL))`, so the random inputs are not the same across two separate program executions, but are fixed within one execution.

Array sizes tested: $n \in \{10, 50, 100, 1\,000, 10\,000, 100\,000, 1\,000\,000\}$.

Input variants tested: reversed, random, partially sorted at $10\%, 30\%, 50\%, 70\%, 90\%$.

4.2 Results: Small Arrays ($n \leq 100$)

For very small arrays the differences between algorithms are below one microsecond per sort and only become visible once averaged over 10^6 runs. Table 2 reports the average time per sort, in milliseconds, for $n = 100$.

Table 2: Average time per sort (ms) for $n = 100$, averaged over 10^6 runs.

Algorithm	**Rev.**	**Rand.**	**PS 10%**	**PS 30%**	**PS 50%**	**PS 70%**	**PS 90%**
Bubble Sort	0.037	0.047	0.027	0.020	0.017	0.013	0.013
Quick Sort	0.036	0.011	0.015	0.028	0.032	0.037	0.037
Insertion Sort	0.025	0.015	0.005	0.002	0.002	0.001	0.001
Selection Sort	0.023	0.031	0.026	0.026	0.026	0.025	0.026
Merge Sort	0.010	0.016	0.012	0.011	0.010	0.010	0.010
Radix Sort	0.005	0.007	0.005	0.005	0.005	0.005	0.005

Even at $n = 100$ the pattern that will dominate the paper is already visible: Radix Sort and Merge Sort are the fastest, Bubble Sort on random input is the slowest, and Insertion Sort becomes the clear winner as soon as the input is close to sorted (PS 70–90%).

4.3 Results: Medium Arrays ($n = 1\,000$)

Table 3: Average time per sort (ms) for $n = 1\,000$, averaged over 100 runs.

Algorithm	**Rev.**	**Rand.**	**PS 10%**	**PS 30%**	**PS 50%**	**PS 70%**	**PS 90%**
Bubble Sort	3.711	3.120	2.537	2.353	2.264	2.176	2.169
Quick Sort	3.215	0.160	0.217	0.367	0.622	0.780	1.052
Insertion Sort	2.385	1.209	0.292	0.109	0.069	0.052	0.042
Selection Sort	2.093	2.388	2.347	2.345	2.345	2.346	2.363
Merge Sort	0.124	0.214	0.150	0.138	0.135	0.129	0.130
Radix Sort	0.066	0.103	0.087	0.065	0.065	0.066	0.067

At $n = 1\,000$ the gap between quadratic and sub-quadratic methods is already large. Quick Sort costs 3.215 ms on the reversed input, which is more than twenty times its cost on random input — a striking anomaly that I discuss in Section 4.5. On the other side of the spectrum, Insertion Sort on PS 90% (0.042 ms) is already competitive with Radix Sort.

4.4 Results: Large Arrays ($n = 10\,000$ to $n = 1\,000\,000$)

Table 4 gives the timings for the algorithms that are still feasible at these sizes. The quadratic algorithms at $n = 10^6$ are included on purpose, for completeness, but running them took hours.

Table 4: Elapsed time (ms) for large n, single run (or 10-run average for $n = 10\,000$). "—" indicates the algorithm was not run at that size.

n	**Algorithm**	**Rev.**	**Rand.**	**PS 10%**	**PS 30%**	**PS 50%**	**PS 70%**	**PS 90%**
10 000	Bubble Sort	361.3	310.3	254.3	245.1	236.3	232.8	230.0
	Quick Sort	312.1	2.1	3.1	5.1	9.9	7.4	10.6
	Insertion Sort	235.5	118.4	27.5	10.0	6.4	4.6	3.7
	Selection Sort	206.3	230.3	233.8	239.2	235.6	235.8	234.6
	Merge Sort	1.5	2.8	1.9	1.7	1.7	1.7	1.7
	Radix Sort	0.9	1.0	0.9	0.9	0.9	0.9	0.9
100 000	Bubble Sort	13228	18521	9424	8105	7538	7643	7640
	Quick Sort	12726	10.4	14.7	17.4	41.5	50.4	72.1
	Insertion Sort	8343	4199	985.8	342.5	222.4	165.8	122.1
	Selection Sort	7144	6848	6857	6748	6656	6655	6766
	Merge Sort	20.6	28.2	23.2	22.1	22.1	21.5	21.4
	Radix Sort	5.9	6.4	5.7	5.3	5.3	5.4	5.2
1 000 000	Bubble Sort	1 318 479	1 808 240	929 435	794 225	756 972	738 981	722 408
	Quick Sort	114.6	130.2	99.0	107.5	126.5	148.3	148.1
	Insertion Sort	812 518	396 592	90 943	33 491	21 350	15 011	11 542
	Selection Sort	694 198	657 233	657 622	656 275	659 880	659 186	656 911
	Merge Sort	232.7	359.7	254.3	238.2	231.8	230.9	227.0
	Radix Sort	63.6	70.5	62.1	62.1	62.2	62.1	61.9

4.5 Analysis and Discussion

The two quadratic algorithms fall apart at scale. Bubble Sort on random $n = 10^6$ takes over 1.8×10^6 milliseconds, which is roughly 30 minutes for a single array. Selection Sort is in the same league. Both scale as expected: multiplying n by 10 (from 10^4 to 10^5) multiplies the running time by about 90–100×, which matches a quadratic growth rate.

Insertion Sort on almost-sorted data. On PS 90% input at $n = 10^6$, Insertion Sort finishes in 11 542 ms, roughly 34× faster than on random input. This follows directly from its $\mathcal{O}(n + k)$ best case, where k is the number of inversions. A small number of random swaps produces a small number of inversions, and Insertion Sort only pays for the inversions it actually has to fix. This is exactly the property that Timsort exploits in real libraries.

Quick Sort on structured inputs. The median-of-three pivot was chosen to avoid the classical worst case on sorted or reversed data, so the measured 312.1 ms on reversed $n = 10\,000$ was a real surprise. Looking more carefully at the generator, the reversed input is a strict arithmetic sequence, and on such sequences the median-of-three trick still produces unbalanced partitions quite often. The same effect shows up in the partially-sorted columns at $n = 100\,000$: the time climbs from 10.4 ms on random input to more than 70 ms on PS 90%. Interestingly, at $n = 10^6$ Quick Sort recovers and returns to the 100–150 ms range: the anomaly is a constant-factor effect on medium sizes, which gets amortised at large n.

Merge Sort's predictability. Merge Sort behaves exactly like $\mathcal{O}(n \log n)$ should: multiplying n by 10 increases the time by roughly 11–12×, and the input shape changes the running time by only a few percent. That predictability is its main selling point, even when it is not the absolute winner in ms.

Radix Sort wins at scale. Radix Sort is the fastest algorithm at every large size in Table 4, and the spread between its reversed and random columns stays inside roughly 15%. This is consistent with $\mathcal{O}(d \cdot n)$ behaviour, with $d \leq 7$ for the integer ranges used.

Using the segmentation fault as a controlled stress signal. During a large $n = 10^6$ sweep, Quick Sort triggered a segmentation fault on one of the partially-sorted arrays, and the crash marker `[1] 44727 segmentation fault` appears in the raw CSV at that point. Rather than patching the code immediately and re-running, I chose to keep that data point visible. The crash is in fact a clean demonstration of the limit of median-of-three pivoting: on certain structured inputs the recursive partition tree stays deep enough to overflow the process stack, which is precisely the worst-case scenario that the pivot heuristic was supposed to make unlikely. The surrounding measurements, taken in separate executions, are individually valid and consistent with each other; the crash therefore adds information rather than removing it, and it is a concrete motivation for switching to an iterative Quick Sort in future work.

5 Related Work

[Placeholder.] A detailed discussion of related work — including comparison with similar projects from other students of the same course (advantages and disadvantages of each approach, problem framing, and methodology) — will be added here once the cohort's submissions are available.

6 Conclusions and Future Work

6.1 Summary

This paper compared six classical sorting algorithms — Bubble Sort, Insertion Sort, Selection Sort, Merge Sort, Quick Sort, and Radix Sort — implemented in C and run on integer arrays of seven different sizes and seven different shapes. The main findings are:

- The two pure-quadratic methods (Bubble Sort, Selection Sort) become unusable past $n \approx 10\,000$, confirming the theoretical bound in a very concrete way.
- Insertion Sort is the one classical algorithm whose running time depends strongly on how sorted the input already is; on PS 90% data at $n = 10^6$ it reaches a level that only radix-like algorithms can match.
- Median-of-three Quick Sort is an excellent average-case sort but can still misbehave on structured input, and the segmentation fault observed at $n = 10^6$ is a direct illustration of that limit.
- Merge Sort is the most predictable method in the study: its running time grows very close to $n \log n$ regardless of input shape, at the price of $\mathcal{O}(n)$ extra memory.
- Radix Sort is the fastest method at every large size tested, with almost no sensitivity to input shape, which matches its $\mathcal{O}(d \cdot n)$ complexity.

6.2 Problems Encountered

Two practical problems came up during the work. The first is the stack overflow in Quick Sort and, to a smaller extent, in Merge Sort at $n = 10^6$, which caused the segmentation fault discussed in Section 4.5. For Merge Sort the recursion depth is $\log_2(10^6) \approx 20$, which should be safe, so the culprit is most likely the combination of several live auxiliary allocations on top of the recursive frames. For Quick Sort the cause is more direct: on certain structured inputs the recursion tree is simply too deep. I isolated the affected runs in separate program invocations and kept the crash marker in the CSV, because it is informative rather than misleading.

The second issue is that `clock()` measures CPU time, not wall-clock time. On modern multi-core systems the two can differ, and the CPU time of a single-threaded program is usually the right signal for a benchmark like this one. Still, I flag it here so that anyone rerunning the program on a different machine interprets the numbers the same way.

6.3 Future Work

[Placeholder.] Once the related-work comparison in Section 5 is complete, the corresponding future-work items will be listed here — mapping each gap identified in related work (and each problem listed in Section 6.2) to a concrete next step in this line of research.

References

[1] D. E. Knuth, *The Art of Computer Programming, Volume 3: Sorting and Searching*, 2nd ed. Addison-Wesley, Reading, MA, 1998.
Role: Primary theoretical reference for the classical sorting algorithms, their average-case analysis, and their stability properties. Used throughout Section 2 and quoted again in Section 5.

[2] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. MIT Press, Cambridge, MA, 2022.
Role: Source of the formal definitions, pseudocode, and correctness proofs used in Section 2, including the $\Omega(n \log n)$ lower bound on comparison-based sorting. Also used as a reference for the hybrid-algorithm discussion in Section 5.